GRIFFITH COLLEGE DUBLIN

Assignment Cover Sheet

Student name:	**Joshan John**		
Student number:	**3029883**		
Faculty:	**Computing Science**		
Course:	**BSCH-CSP**	**Stage/year:**	**4**
Subject:	**Cloud Services & Platforms**		
Study Mode:	Full time **x**	Part-time	
Lecturer Name:	**Sheresh Zahoor**		
Assignment Title:	**Building an Implementation of Dropbox**		
No. of pages:	**10**		
Disk included?	Yes	No	**x**
Additional Information:	(ie. number of pieces submitted, size of assignment, A2, A3 etc)		
Date due:	**03/05/2026**		
Date submitted:	**26/04/2026**		

Plagiarism disclaimer:

I understand that plagiarism is a serious offence and have read and understood the college policy on plagiarism. I also understand that I may receive a mark of zero if I have not identified and properly attributed sources which have been used, referred to, or have in any way influenced the preparation of this assignment, or if I have knowingly allowed others to plagiarise my work in this way.

I hereby certify that this assignment is my own work, based on my personal study and/or research, and that I have acknowledged all material and sources used in its preparation. I also certify that the assignment has not previously been submitted for assessment and that I have not copied in part or whole or otherwise plagiarised the work of anyone else, including other students.

Signed: joshan john **Date: 26/04/2026**

Please note: *Students **MUST** retain a hard / soft copy of **ALL** assignments as well as a receipt issued and signed by a member of Faculty as proof of submission.*

Table of Contents

1. Introduction *2*

2. Tech Stack *2*

2.1 List of Tech Stack 2

2.2 Dependency management 3

2.3 Data management 3

3. Project Structure *3*

3.1 Naming Conventions 3

3.2 Design Pattern 3

4. Source Code. *5*

4.1 Relevant Functions 5

4.2 Entity Schemas 6

4.3 Frontend User Interactions Control 6

4.4 Application Logs 6

5. Backend Endpoints *7*

Appendix *8*

1. Introduction

Jcloud is a platform-as-a-service application that offers storage services to users. The project is similar to a Dropbox storage service. Jcloud's dedicated user workspace allows file management for registered users.

Source code: https://github.com/joshanjohn/jcloud-pass.git

2. Tech Stack

The project utilizes Python as the core language for this application, with JavaScript for handling some of the client interactions and authentications. The application is based on Python's FastAPI framework for backend and Jinja2 templating for web rendering. It also uses HTML and CSS as web technologies.

2.1 List of Tech Stack

- Python
- UV (Python dependency manager)
- pydrantic (validations)
- Fast API (Python framework)
- Jinja2 (templating)
- HTML
- CSS
- Tailwind CSS (inline CSS)
- JavaScript
- Firebase (authentication)
- Azure (file storage)
- MongoDB (record management)
- UNPKG (icons)
- Google Fonts - Inter (font style)

2.2 Dependency management

The Python backend dependencies are mainly managed using the **UV**[1] dependency manager (written in Rust), which allows faster parallel downloads and automatically creates a dedicated Python environment. All project configurations are written in the "**pyproject.toml**" file.

2.3 Data management

The project data is stored across various services. Firebase is used for user authentication and authorization. The workspace directories and folder information are stored in MongoDB clusters. Plus, Azure for file data or blob storage.

3. Project Structure

The project code base is well-structured for maintainability and scalability, and follows a clean code practice and proper naming conventions. All the project secrets are safe and secure. The source code itself follows a loosely coupled software design for easier modification and fault tolerance.

3.1 Naming Conventions

Code base follows snake case naming convention for functions, files, and folders. But the class names follow Pascal case naming. The function names are descriptive and contain docstrings and comments.

3.2 Design Pattern

Since the application is structured in a monolithic manner, both the frontend and backend are placed inside the /***src*** folder. The business logic or system implementations follow a factory interface programming. Where each low-level service implements corresponding interface classes. So that, even though the tech stack changes or is modified, its functionalities or methods will be the same. Plus, it is easier to include more high-level classes and functionalities (e.g., user management service) into the system service.

[1] https://docs.astral.sh/uv/

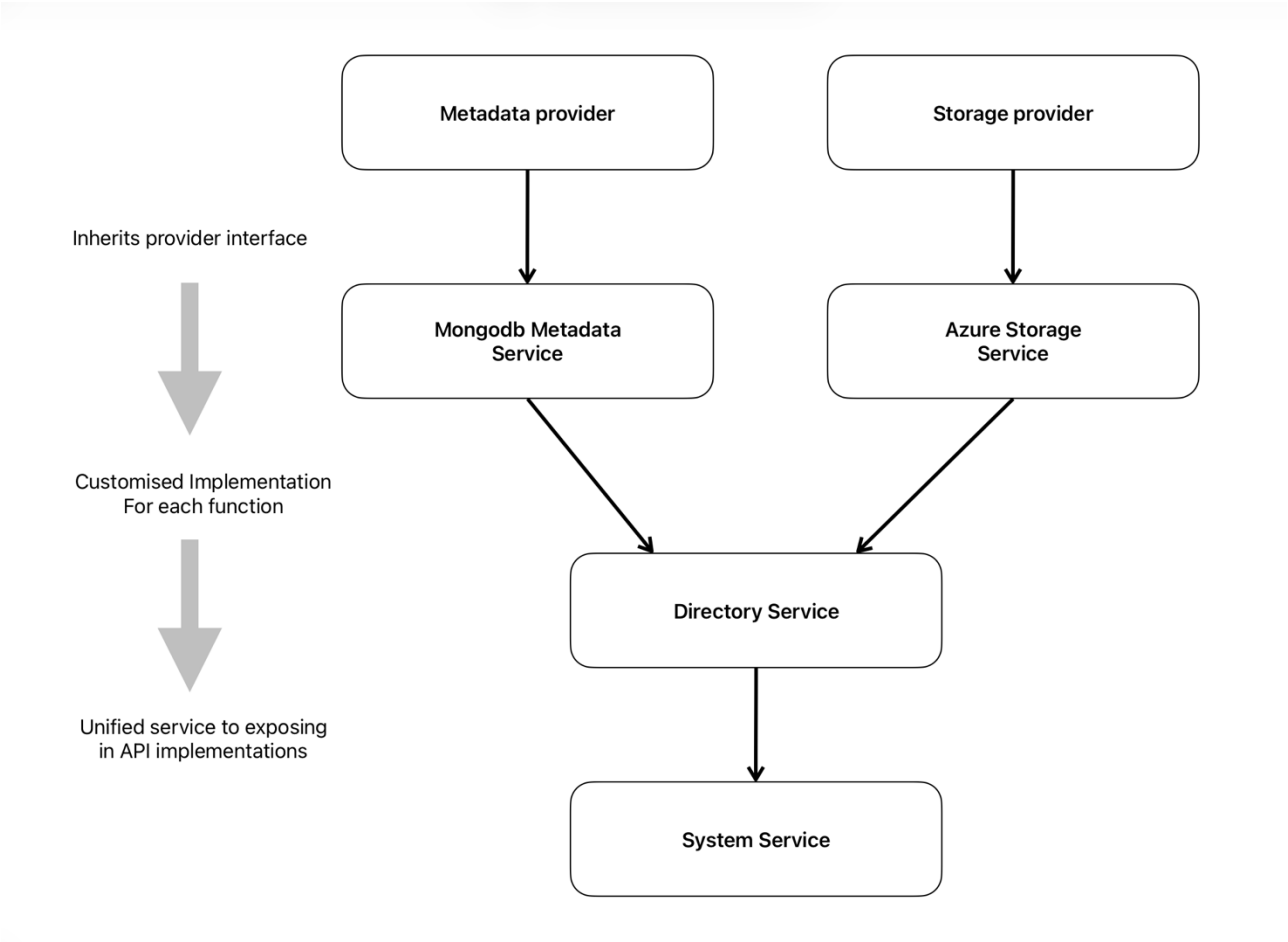


Image of service hierarchy from the interface to the high-level service.

Here, the provider are interfaces with abstract methods. These methods will be implemented accordingly by various low-level services. Currently, the cloud metadata (structure) is handled by MongoDB, and the actual file data storage is done using Microsoft's Azure blob containers. So even in the future we decide to upgrade to a different technology to maintain the metadata and storage, the functionality won't change, thus loose coupling is achieved. The low-level functions are then used to make up a directory service, where full file management functionalities are performed. Then the high-level service is inherited by the main system service.

The frontend web programs are organized in ***templates*** and ***static*** files. The application APIs are placed inside the ***routes*** folder. The entity models are organized in the ***entities*** folder. The basic connection object creation is placed in the ***database/core*** folder. All the service classes are in the ***services*** folder, and their interfaces on ***factory*** folder. In addition, all the other helpers, constants, and logger are in the ***utils*** folder.

```
src/
├── database/core
├── entities
├── factory
├── routes
├── services
├── static
├── templates
└── utils
```

Image of folder structure inside "src" folder

4. Source Code.

4.1 Relevant Functions

As the low-level services implement the provider interfaces, which are utilized by high-level classes. In the current application, the directory service contains most of the high-level code for maintaining complete file and storage management.

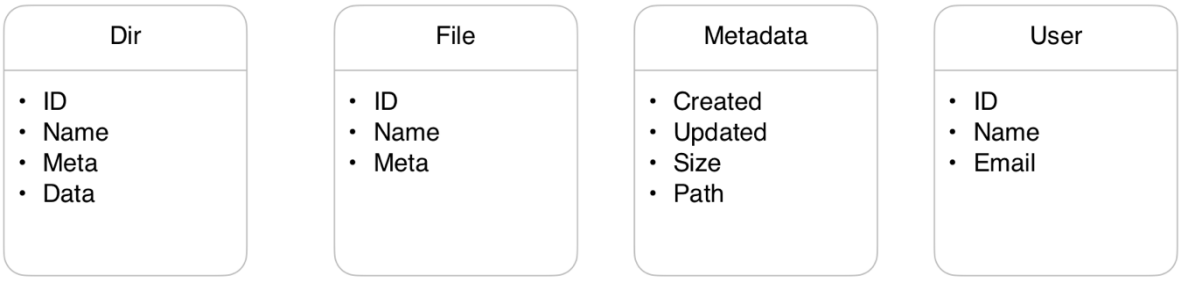
The following methods from the directory service are mostly used across the endpoints.

Function Name	Description
get_all_dir()	Fetch all directory information for the signed user.
get_dirs_in_path()	Fetch all directory information inside a path location.
get_files_in_path()	Fetch all file inside a path location.
get_duplicate_files()	Fetch duplicated files inside entire cloud system.
upload_file()	Upload a file into cloud.
create_dir()	Create a directory in cloud system.
delete_dir()	Delete a directory from cloud system.
delete_file()	Delete a file from cloud system..
check_file_exists()	Check if file exists in metadata.

download_file()	Download file form cloud system.

4.2 Entity Schemas

As a clean code practice, all the entity properties are made into schemas or models with **Pydantic**[2] **BaseModel** for better data validation and usability. There are directory, file, metadata, and user entities. A file entity has a file ID, name, and metadata.



Object diagram of all entity models.

A directory entity has an ID, a name of the directory, metadata, and file data. Metadata contains the created date, updated date, size, and location path. Similarly, the user has an ID, name, and respective email.

4.3 Frontend User Interactions Control

JavaScript form data is used for performing most of the in-workspace actions. Which is in the **workspace.js** file. The **firebase-auth.js** also handles the user login and signup with a form, and it also authenticates and stores the token as cookies from Firebase for successful login.

4.4 Application Logs

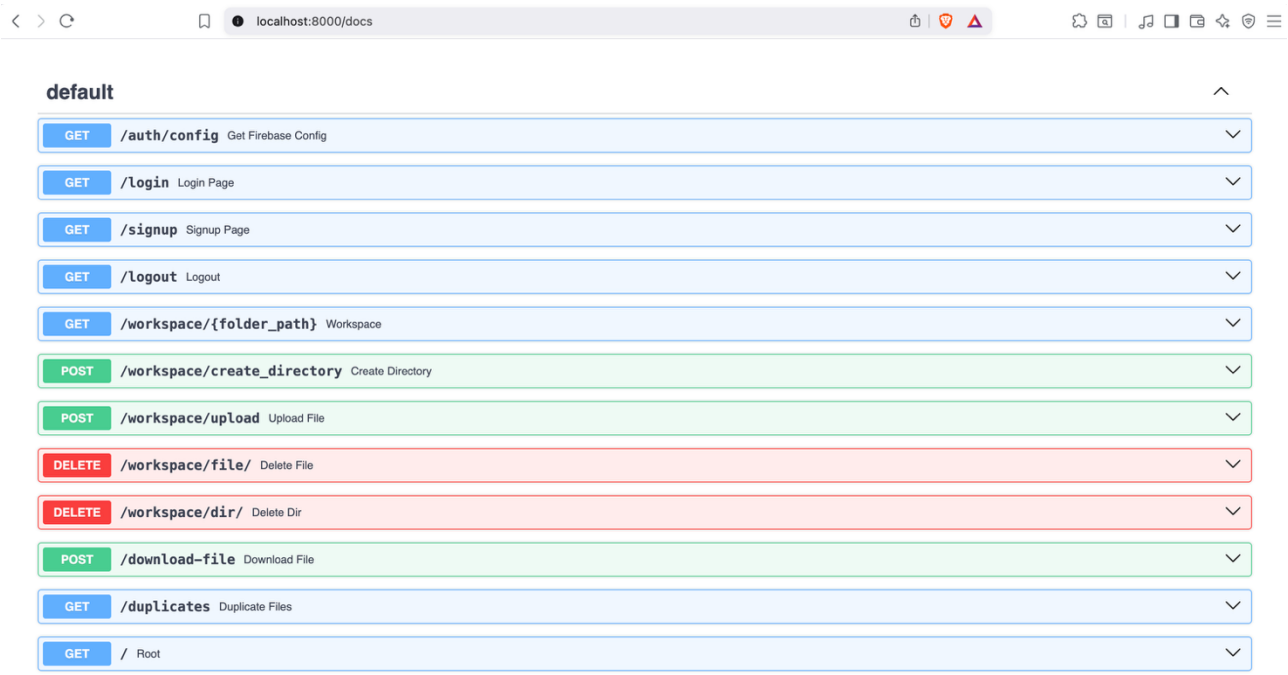
On running the application and Azure service, both of their logs are saved into the .logs folder in the project folder as app.log and azure.log files. This will help to record and analyse what is happening

[2] https://pydantic.dev/docs/

when the user interacts with the application. In case of bugs or errors, these logs serve as the main source of identifying the error so that debugging them becomes much easier.

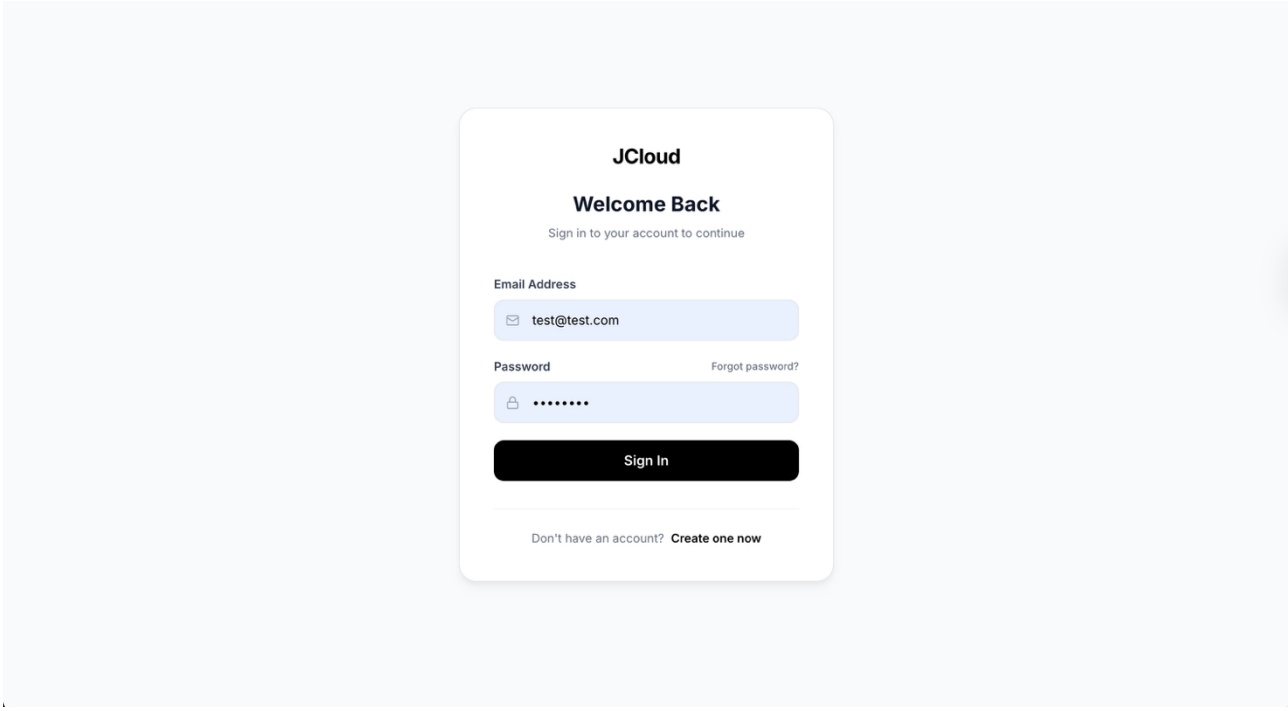
5. Backend Endpoints

Backend endpoints developed using Python FastAPI, which return either JSON or an HTML response. A full list of all the endpoint documentation is available at the **/docs** page. Developers can also test the API's on the same page.

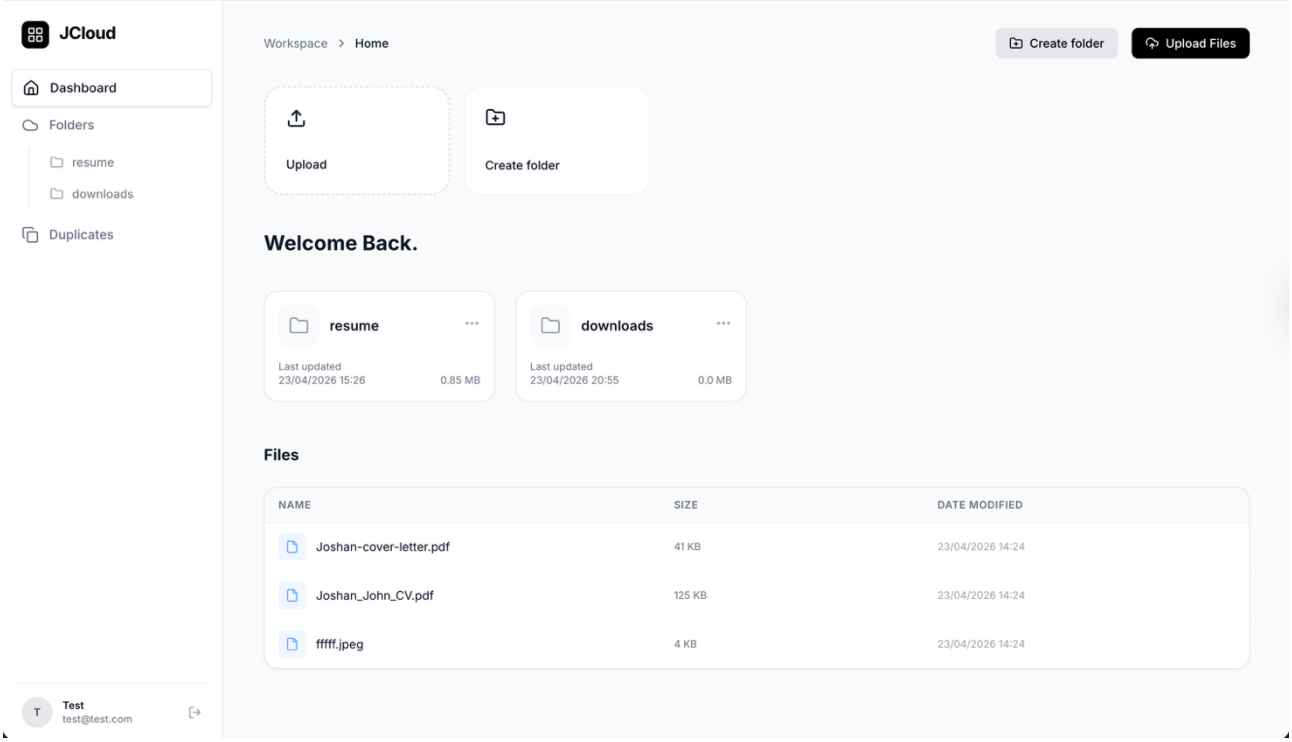


Appendix

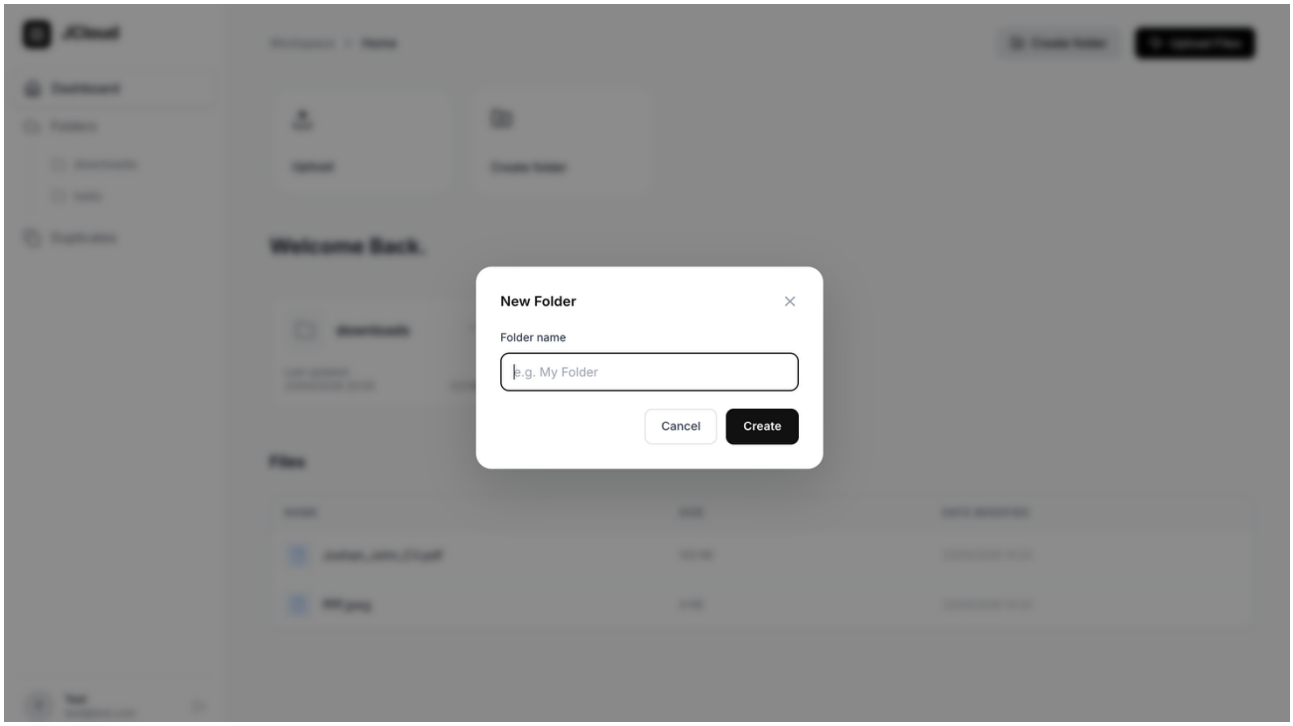
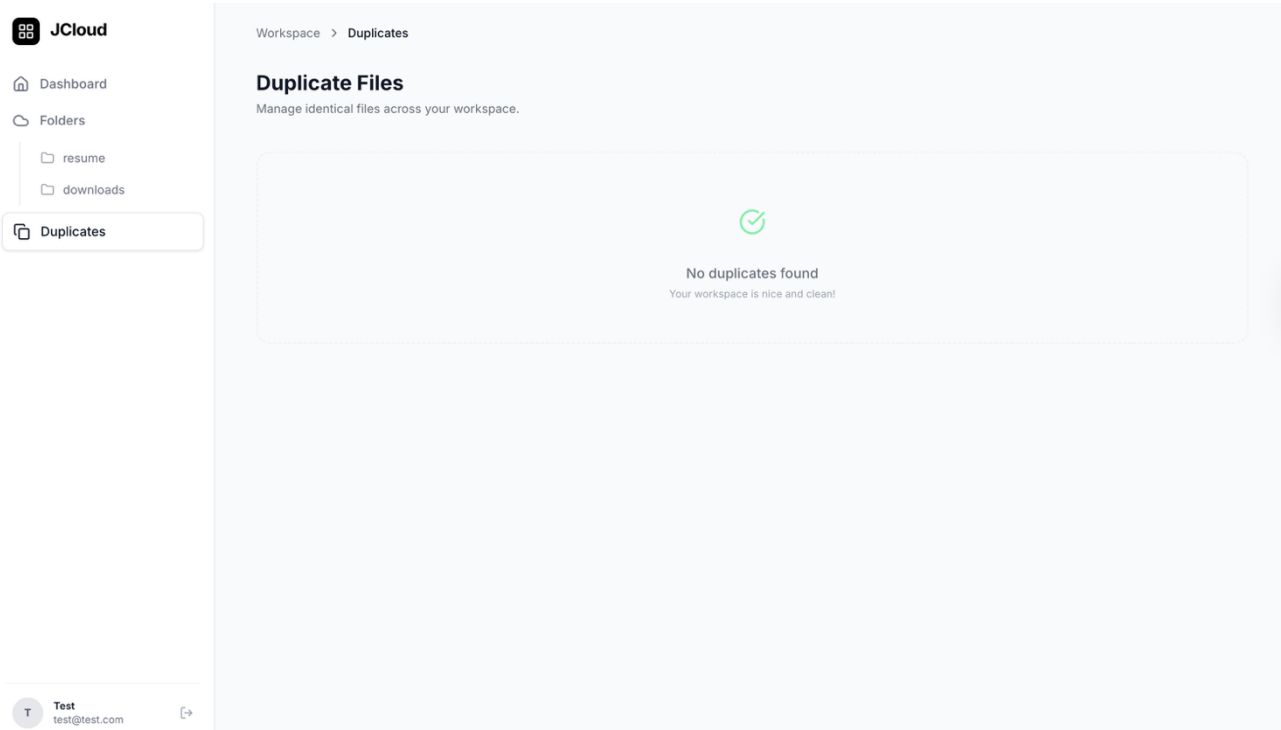
1. App login Ui



2. App workspace home page UI.



3. Create folder modal



4. App workspace duplicate page UI